Actúa como analista y documentador de bases de datos. Tu tarea es elaborar el DICCIONARIO DE DATOS completo de la base de datos `db_test` (MySQL 8.0.36) a partir del archivo `BD13.sql`, que es un mysqldump.

Reglas de lectura del archivo

- Si el archivo contiene datos, casi todos son sentencias `INSERT INTO` de `tb_logs` (y algunos de tb_users, tb_products y tbc_categories). NO leas ni imprimas esos INSERT: pueden contener correos y datos personales reales. Trabaja solo con CREATE TABLE, CREATE VIEW, CREATE TRIGGER y CREATE PROCEDURE (usa grep/Python para extraerlos).

- No modifiques `BD13.sql`. Escribe todo en una carpeta nueva `/diccionario_datos`.

- No inventes información. Si algo no se puede deducir del SQL, escribe "No especificado" y lístalo al final en una sección "Pendientes por confirmar".

Objetos que debes documentar

- 20 tablas: tb_buyer_orders, tb_carts, tb_logs, tb_orders, tb_payments, tb_products, tb_user_addresses, tb_users, tbc_categories, tbc_payment_methods, tbc_payment_methods_types, tbd_address_components, tbd_carts_items, tbd_order_details, tbd_order_items, tbd_products_categories, tbd_products_users_providers, tbd_user_payment_methods, tbd_users_wishlist_products, tbd_wishlist.

- 18 vistas `vw_trazabilidad_*`.

- 57 triggers (`trg_*`, AFTER INSERT/UPDATE/DELETE).

- 3 procedimientos: sp_soft_delete, sp_soft_delete_product, sp_soft_delete_user.

Contenido del diccionario

1. Por cada tabla

a) Ficha: nombre, prefijo/tipo (principal tb_, catálogo tbc_, detalle tbd_), motor, charset/collation, descripción de su propósito de negocio (1-2 líneas, en español).

b) Tabla de columnas con: #, nombre, tipo de dato, longitud/precisión, ¿acepta NULL?, valor por defecto, extras (AUTO_INCREMENT, GENERATED STORED, ON UPDATE CURRENT_TIMESTAMP), tipo de llave (PK/FK/UK) y descripción en español. Para ENUM lista todos los valores posibles. Para `bit(1)` `status` indica que 1 = activo y 0 = inactivo (borrado lógico).

c) Restricciones: PK (simple o compuesta), UNIQUE, FOREIGN KEY (columna → tabla.columna, ON DELETE/UPDATE) y CHECK con su expresión.

d) Índices.

e) Relaciones: tablas que referencia y tablas que la referencian, con cardinalidad (1:1, 1:N, N:M).

f) Triggers asociados.

2. Vistas

Por cada vista: nombre, propósito, tablas base, columnas que expone y tipo de dato/origen.

3. Triggers

Una tabla resumen: nombre, tabla, evento (INSERT/UPDATE/DELETE), momento (AFTER), y qué registra en `tb_logs` (columnas que llena y formato del campo `description`).

4. Procedimientos almacenados

Parámetros (IN/OUT, tipo), qué hace paso a paso, tablas que afecta y ejemplo de llamada.

5. Catálogo de dominios

Lista consolidada de todos los ENUM, y de los campos `status` (bit vs tinyint) indicando en qué tablas se usa cada uno.

6. Convenciones de nombres

Explica los prefijos (tb_, tbc_, tbd_, vw_, trg_, sp_, fk_, uk_, chk_) y el patrón de columnas de auditoría (creation_date, last_update, status).

7. Observaciones de calidad del diseño (importante)

Revisa y reporta, sin corregir el SQL, al menos:

- `tbd_products_categories` tiene `category_ID` sin FK hacia `tbc_categories`.

- Posible duplicidad funcional: `tb_orders`/`tbd_order_details` vs `tb_buyer_orders`/`tbd_order_items`; y `tbd_wishlist` vs `tbd_users_wishlist_products`.

- `tb_buyer_orders` solo tiene `seller_ID`, no identifica al comprador.

- `tbd_carts_items` y `tbd_order_items` referencian `user_ID` y `product_ID` de `tbd_products_users_providers` por separado, en lugar de la PK compuesta.

- Inconsistencias de tipos de datos entre llaves relacionadas (INT vs INT UNSIGNED vs BIGINT) y entre `status` (bit(1) vs tinyint).

- Verifica que no quede ninguna referencia a `tbd_shopping_cart` ni a `tbd_users_offer_products` en tablas, vistas, triggers o procedimientos. Si no hay, indícalo como "Verificado: sin referencias huérfanas".

- Cualquier otra inconsistencia que encuentres.

Clasifica cada hallazgo como Alta/Media/Baja y propón una recomendación.

Entregables (en /diccionario_datos)

1. `diccionario_datos.md`: documento principal con todo lo anterior, con índice navegable.

2. `diccionario_datos.xlsx`: hojas "Tablas", "Columnas", "Relaciones", "Vistas", "Triggers", "Procedimientos", "Dominios_ENUM", "Observaciones".

3. `diagrama_er.mmd`: diagrama entidad-relación en Mermaid (erDiagram) con las 20 tablas, sus PK/FK y cardinalidades. Si es muy grande, divídelo por módulos: Usuarios y direcciones, Productos y categorías, Carritos y wishlist, Órdenes y pagos, Auditoría.

4. `schema.dbml`: el mismo modelo en formato DBML (útil para importarlo en herramientas de diagramado como Archify).

5. `README.md`: qué se generó y cómo regenerarlo.

Forma de trabajo

1. Primero extrae la estructura del SQL con un script (sin leer los INSERT) y guárdala como JSON intermedio (`estructura.json`).

2. Genera los entregables a partir de ese JSON, no a mano, para evitar errores.

3. Valida al final: que el conteo sea 20 tablas, 18 vistas, 57 triggers y 3 procedimientos, y que cada FK del SQL aparezca en el diccionario. Muéstrame el resultado de la validación.

4. Redacta en español, con tono técnico y claro. Los nombres de objetos se mantienen tal cual (en inglés).

Amplía el diccionario de datos de `db_test` que generaste en /diccionario_datos. Modifica y reutiliza tus scripts (extraer.py, generar.py, generar_modelos.py) y estructura.json; no empieces de cero ni toques BD13.sql. Sigue sin leer los INSERT.

Requisito del profesor

El diccionario debe incluir, para CADA TABLA y CADA COLUMNA (20 tablas, todas sus columnas), lo siguiente:

A) Descripción general detallada

- Por tabla: qué entidad o proceso de negocio representa, a qué módulo pertenece (usuarios, productos, carritos, órdenes, pagos, auditoría), cómo se relaciona con otras tablas, reglas de negocio visibles (CHECK, UNIQUE, ON DELETE, columnas generadas) y qué triggers tiene. Mínimo 4-5 líneas por tabla.

- Por columna: qué dato almacena, para qué se usa, cómo se genera (autoincremental, por defecto, calculado, capturado por el usuario) y restricciones que la afectan. Mínimo 1-2 oraciones, nunca solo repetir el nombre.

- Básate en nombres, tipos, constraints y los mensajes de los triggers. Si una descripción es una inferencia y no algo explícito en el SQL, márcala con "(inferido)" y lístala en "Pendientes por confirmar".

B) Clasificación por columna (una fila por columna)

1. Valor: Monovalor o Multivalor. Toda columna relacional es monovalor; si el atributo conceptual es multivalor (p. ej. categorías de un producto, componentes de una dirección), indícalo como "Multivalor resuelto mediante tabla X". Si una columna de texto concentra varios datos (p. ej. `tb_logs.description`, `tb_orders.shipping_address`), márcalo como "Monovalor (contiene datos compuestos en texto libre)".

2. Origen: Almacenado o Derivado. Derivado = columna GENERATED o calculable de otras (indica la fórmula). Si es almacenado pero derivable (p. ej. `tb_orders.total_amount`, `tb_buyer_orders.subtotal`), escribe "Almacenado (derivable de ...)".

3. Obligatoriedad: Obligatorio (NOT NULL) u Opcional (NULL). Indica si el valor lo aporta el usuario o el sistema (DEFAULT, AUTO_INCREMENT).

4. Llave: PK, PK compuesta (indica las columnas que la forman), FK (a qué tabla.columna), UK, o No.

5. Naturaleza: Cualitativa nominal, Cualitativa ordinal, Cuantitativa discreta o Cuantitativa continua. Las fechas se clasifican como Cuantitativa (temporal). Los ENUM con orden lógico (p. ej. estados de una orden) son ordinales; sin orden, nominales.

6. Composición en notación formal (ver más abajo).

7. Dominio: tipo, longitud/precisión, conjunto o rango de valores válidos y formato, usando notación de conjuntos: `status ∈ {0,1}`, `quantity ∈ $\mathbb{Z}^+$`, `unit_price ∈ $\mathbb{R}$, ≥ 0, 2 decimales`, `fulfillment_status ∈ {'Pending','Accepted',...}`, `latitude ∈ [-90, 90]`, `email ∈ cadena(≤80), único`. Incorpora los CHECK reales del SQL.

C) Notación formal de composición

Usa la notación de diccionario de datos (DeMarco/Yourdon):

- `=` "se compone de" `+` "y" `[ a | b ]` "uno de" `{ x }` "repetición" `( x )` "opcional" `* texto *` comentario

- Por TABLA: `tb_users = @ID + email + nickname + password + creation_date + (last_login) + (status)`, marcando la PK con `@` y las FK con `#` (o la convención que definas y explicas en una leyenda).

- Por COLUMNA compuesta o con estructura interna, descompón. Ejemplos: `email = usuario + "@" + dominio`; `coordenadas = latitude + longitude` (ambas nulas o ambas con valor, según chk_coordinates); `transaction_reference = {carácter alfanumérico}`.

- Para columnas simples, escribe el tipo atómico: `ID = {dígito}` con su dominio.

- Incluye una sección "Leyenda de la notación" y otra "Criterios de clasificación" con las reglas de arriba, para que el lector entienda cómo se clasificó.

Entregables (actualiza los existentes)

1. `diccionario_datos.md`: por cada tabla, la descripción detallada, la notación de la tabla, y DOS tablas de columnas: (a) Técnica: #, nombre, tipo, longitud, NULL, default, extras, llave; (b) Clasificación: nombre, valor, origen, obligatoriedad, llave, naturaleza, composición formal, dominio, descripción detallada. Mantén el índice navegable y las secciones de vistas, triggers, procedimientos, dominios, convenciones y observaciones.

2. `diccionario_datos.xlsx`: hoja "Columnas" con TODAS las columnas de clasificación como encabezados, una fila por columna; hoja "Tablas" con descripción detallada y notación formal; conserva las demás hojas. Aplica filtros, encabezado fijo y ajuste de texto.

3. `estructura.json`: agrega los nuevos campos para que el sitio web se pueda generar desde aquí.

4. Actualiza `README.md`.

5. Agrega una hoja o sección "Resumen estadístico": cuántas columnas hay por naturaleza, por obligatoriedad, derivadas vs almacenadas, y cuántas PK/FK/UK.

Validación final (muéstrala)

- Ninguna columna sin clasificar: cada columna debe tener los 7 campos llenos (nada vacío).

- El número total de columnas del diccionario coincide con el del SQL.

- Las columnas derivadas encontradas (mínimo `subtotal` en tbd_order_details y tbd_order_items).

- Las PK compuestas se listan completas.

- Lista de descripciones marcadas "(inferido)".

- Idioma: español; nombres de objetos tal cual.

Revisé diccionario_datos/diccionario_datos.md y NO cumple lo que pedí. Ejemplo: la tabla `tb_logs` solo tiene una descripción de una línea, todas las columnas dicen "No especificado en SQL.", y no hay tabla de clasificación ni notación formal. Corrige generar.py y vuelve a generar todo.

Para CADA tabla el .md debe tener:

1. Descripción general detallada (mínimo 4-5 líneas): entidad que representa, módulo, relaciones, reglas de negocio (CHECK, UNIQUE, ON DELETE, columnas generadas) y triggers.

2. Notación formal de la tabla (DeMarco/Yourdon: = + [|] { } ( )), marcando PK y FK.

3. Tabla técnica de columnas (como la actual), pero con "Sin default" en vez de "No especificado" cuando la columna no tiene DEFAULT.

4. Tabla de clasificación, una fila por columna: valor (mono/multivalor), origen (almacenado/derivado, con fórmula), obligatoriedad (obligatorio/opcional), llave (PK/PK compuesta/FK/UK/No), naturaleza (cualitativa nominal u ordinal, cuantitativa discreta o continua), composición en notación formal, dominio (notación de conjuntos, con los CHECK reales) y descripción detallada (1-2 oraciones, nunca "No especificado").

Si una descripción es inferida, márcala "(inferido)".

Incluye al inicio las secciones "Leyenda de la notación" y "Criterios de clasificación".

Validación: no uses python -c con comillas complicadas; escribe un script validar.py. Debe comprobar, por cada tabla del .md, que existan las secciones 1 a 4, que ninguna celda de descripción diga "No especificado", y que el total de filas de clasificación sea 141. Hazlo también para las hojas del XLSX. Muéstrame el resultado y el contenido final de `tb_logs` y `tb_orders` como ejemplo.

Corrige la clasificación en generar.py y regenera md, xlsx y estructura.json:

1. Todo identificador (PK y FK numéricas, ID, *_ID) es "Cualitativa nominal", no cuantitativa. Cuantitativa solo para cantidades, importes, coordenadas y fechas.

2. En "Composición formal" usa notación DeMarco/Yourdon sin tipos de SQL (por ejemplo `ID = {dígito}`), y elimina el texto entre asteriscos que repite el dominio.

3. El dominio de un AUTO_INCREMENT es $\mathbb{Z}^+$ (no incluye el 0).

4. En tb_users.password aclara que almacena un hash.

Ejecuta validar.py de nuevo y muéstrame las filas de tb_users y tbd_order_items.

Ajusta generar.py y regenera md, xlsx y estructura.json:

1. En el dominio de toda FK reemplaza "Identificador FK numérico" por la pertenencia al conjunto de la tabla referenciada, con la regla de borrado. Ejemplo: `buyer_order_ID ∈ {tb_buyer_orders.ID}; ON DELETE sin acción`.

2. En el dominio de las fechas indica el tipo y el default. Ejemplo: `creation_date ∈ DATETIME; default CURRENT_TIMESTAMP`.

3. Ejecuta validar.py y muéstrame la fila de tbd_order_items.subtotal y la de tbd_order_details.subtotal, donde se vea Origen = Derivado y la fórmula.

Revisa todas las descripciones de columnas en generar.py / estructura.json. Hay descripciones de relleno, por ejemplo tb_payments.transaction_reference y tb_payments.payment_date dicen "Almacena X de `tabla` para describir o relacionar ese registro (inferido)", que solo repite el nombre.

1. Detecta todas las descripciones que sigan esa plantilla genérica y lístalas (tabla.columna).

2. Reescríbelas con significado de negocio real, 1-2 oraciones: qué dato guarda, cómo se origina (capturado, calculado, por defecto o generado por el sistema), por qué es obligatorio u opcional y cómo se usa. Ejemplo: transaction_reference = identificador que devuelve la pasarela o el banco para rastrear el pago; es opcional porque un pago en efectivo no lo tiene.

3. Quita de la descripción el texto "Restricciones: ..." porque ya está en las columnas Tipo, NULL y Llave.

4. Si no puedes deducir el significado con certeza, escríbelo con prudencia y déjalo en "Pendientes por confirmar".

5. Regenera md, xlsx y estructura.json, actualiza validar.py para que falle si queda alguna descripción con la plantilla genérica, y muéstrame el resultado y 5 ejemplos reescritos.

Convierte la carpeta `diccionario_datos/` en un sitio estático para GitHub Pages: solo HTML, CSS y JS, sin build ni servidor; librerías por CDN con versión fija.

Entradas (no las modifiques): estructura.json, diccionario_datos.md, diagrama_er.mmd, schema.dbml, diccionario_datos.xlsx.

Crea `/docs` con:

- `index.html`: portada con db_test, MySQL 8.0.36 y conteos (20 tablas, 18 vistas, 57 triggers, 3 procedimientos), y navegación a cada sección.

- Una página o sección por tema: Tablas, Vistas, Triggers, Procedimientos, Dominios y ENUM, Convenciones, Leyenda de la notación y criterios de clasificación, Resumen estadístico, Observaciones de calidad.

- En cada tabla: descripción detallada, notación formal, tabla técnica y tabla de clasificación completa, con filtros por naturaleza, obligatoriedad y origen (derivado/almacenado).

- Diagrama ER con Mermaid.js, con zoom o scroll, dividido por módulos si es grande.

- Buscador de tablas y columnas, menú lateral fijo, modo claro/oscuro, diseño responsivo (tablas con scroll horizontal).

- Botones de descarga del xlsx, md y dbml (cópialos a /docs/descargas).

- `.nojekyll`, y TODAS las rutas relativas (sin "/" inicial).

Reglas:

- Genera las páginas con `generar_sitio.py` a partir de estructura.json, no a mano.

- Sin datos reales: verifica con grep que no aparezca "@utxicotepec" ni hashes ("$2") en /docs.

- Crea `.gitignore` con backups/, *.sql, .archify/, __pycache__/.

- Al final dime cómo probarlo en local y qué archivos subir.